

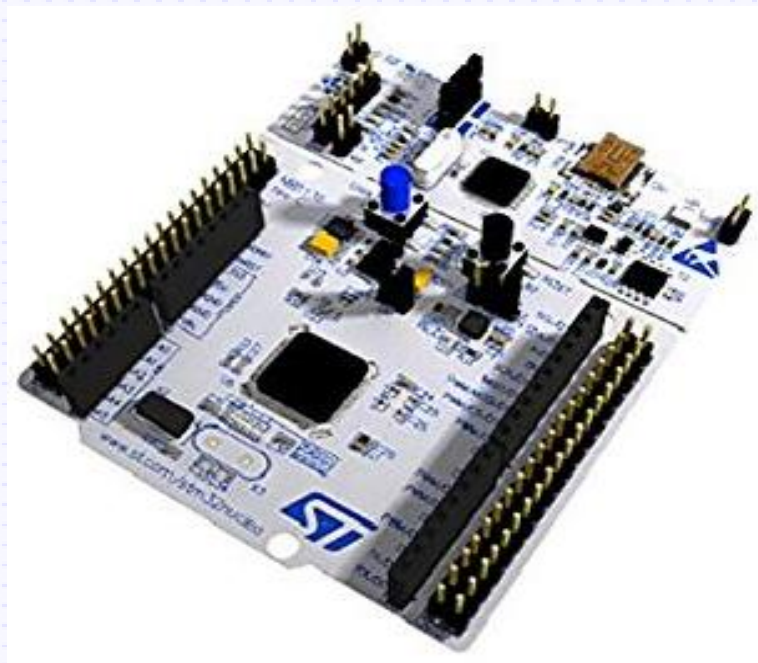
Chapitre I:

Notions de base de la programmation C embarqué



I- Introduction

C'est quoi un système embarqué ?

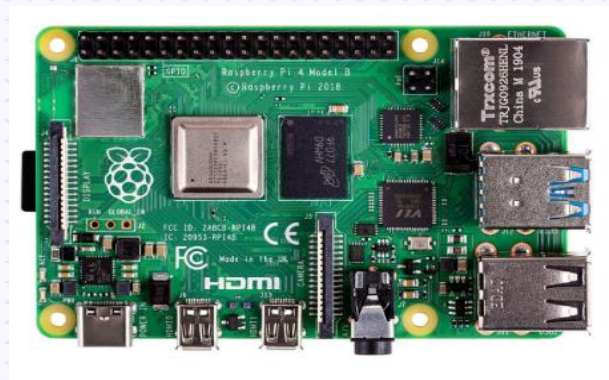


- Un **système embarqué** est défini comme un système **électronique** et **informatique** autonome spécialisé dans une tâche bien précise.

I- Introduction

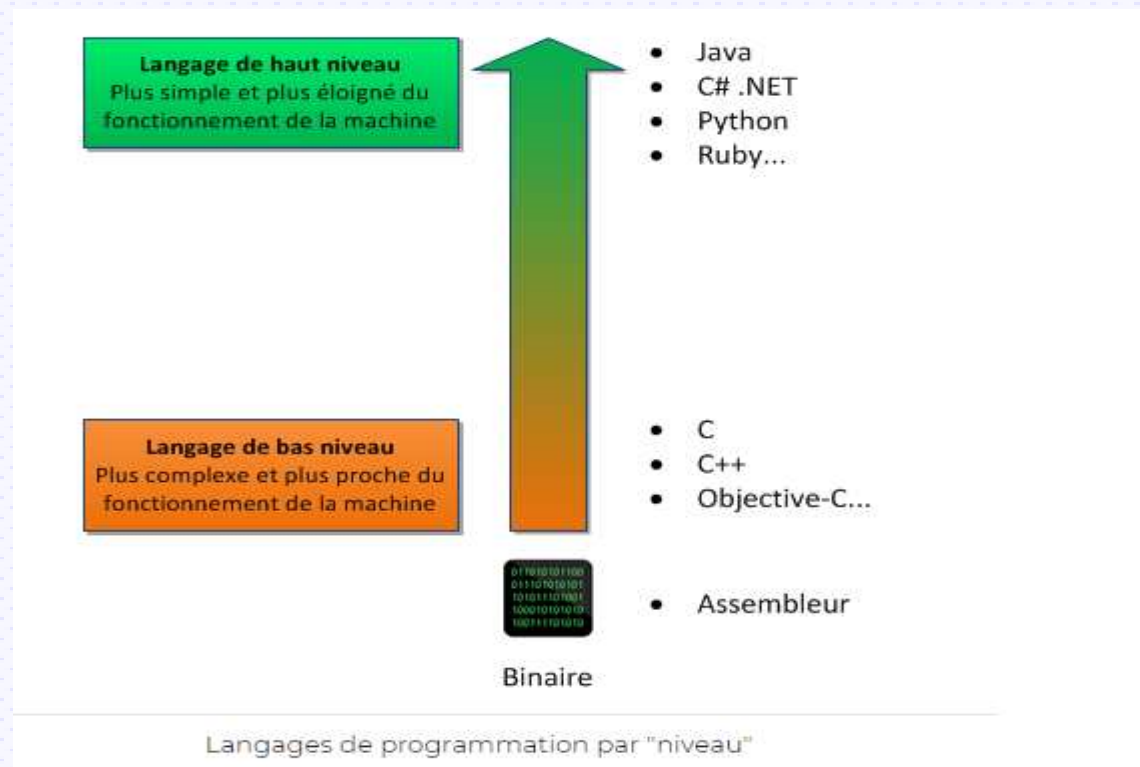
Les plateformes les plus répandus dans les systèmes embarqués:

- Les microcontrôleurs (STM32, PIC, Atmel ...)
- Raspberry PI
- Les Smartphones (Samsung, Iphone, Huawei)
- Les circuits FPGA (Spartan 3E, Artix 7 ...)



II- Les notions de base de la programmation C

Pourquoi le langage C est le plus populaire pour la programmation des microcontrôleurs ?



- Le langage C est proche de la machine
- Les IDE qui utilise le C deviennent très sophistiqués

II- Les notion de base de la programmation C

C'est quoi le C embarqué ?



Le langage C embarqué est le langage C en prenant en considération les contraintes Hardware qui sont:

- Taille de la mémoire (KO vs GO)
- Fréquence de système (MH vs GH)
- Architecture cible (architecture spécifique)

II- Les notions base de la programmation en C

1- Structure d'un programme

- En langage C le **programme principal** et les **sous programmes** sont définies comme des **fonctions**.
- La fonction particulière nommée **main()** est **la fonction principale** d'un « **programme C** »

Syntaxe

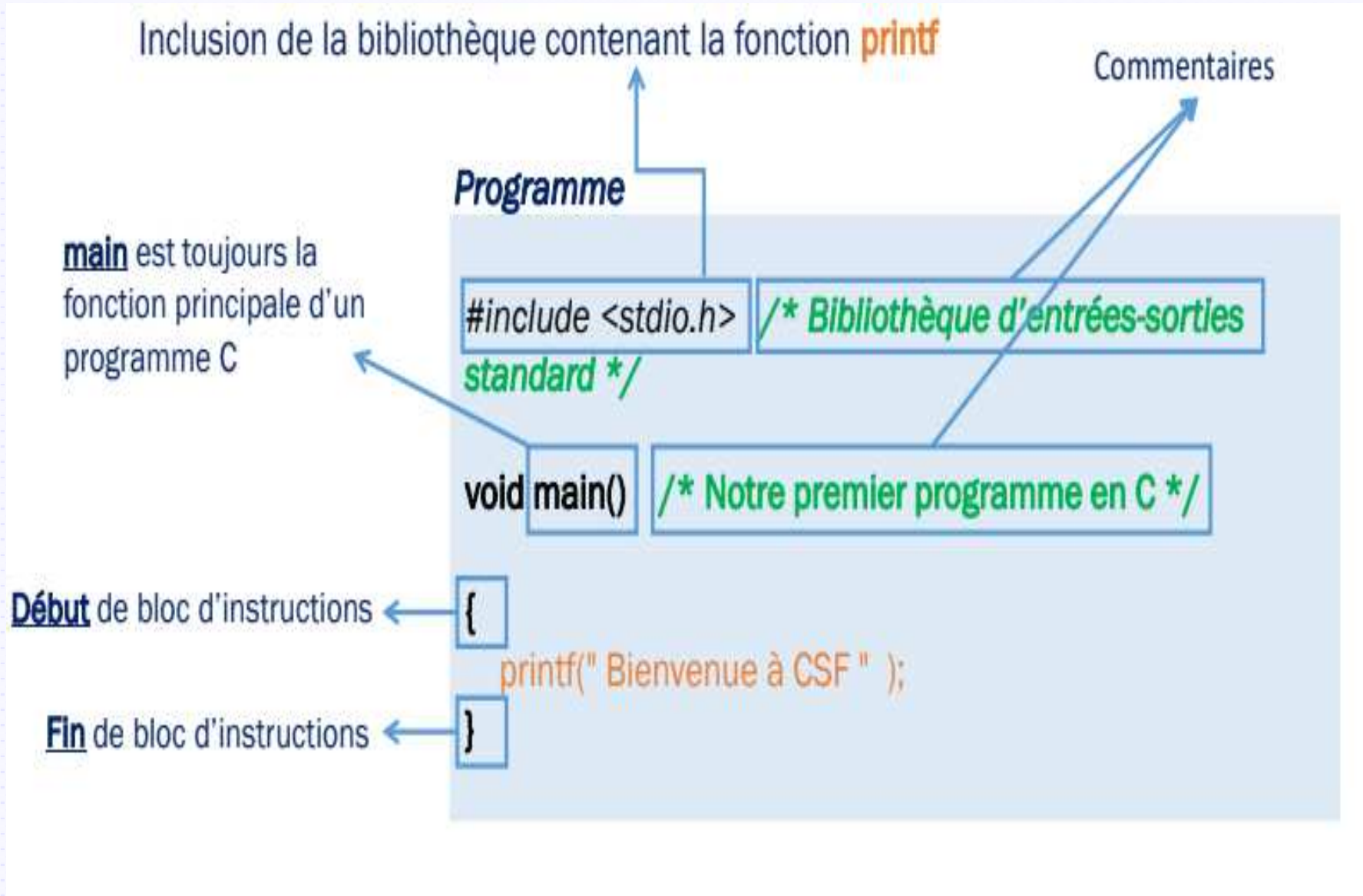
```
<include des  
bibliothèques>  
  
void main()  
{  
    < Traitements >;  
}
```



Programme

```
#include <stdio.h> /* Bibliothèque d'entrées-  
sorties standard */  
  
void main() /* Notre premier programme en C */  
{  
    printf(" Bienvenue à C.S.F" );  
}
```

II- Les notions base de la programmation en C



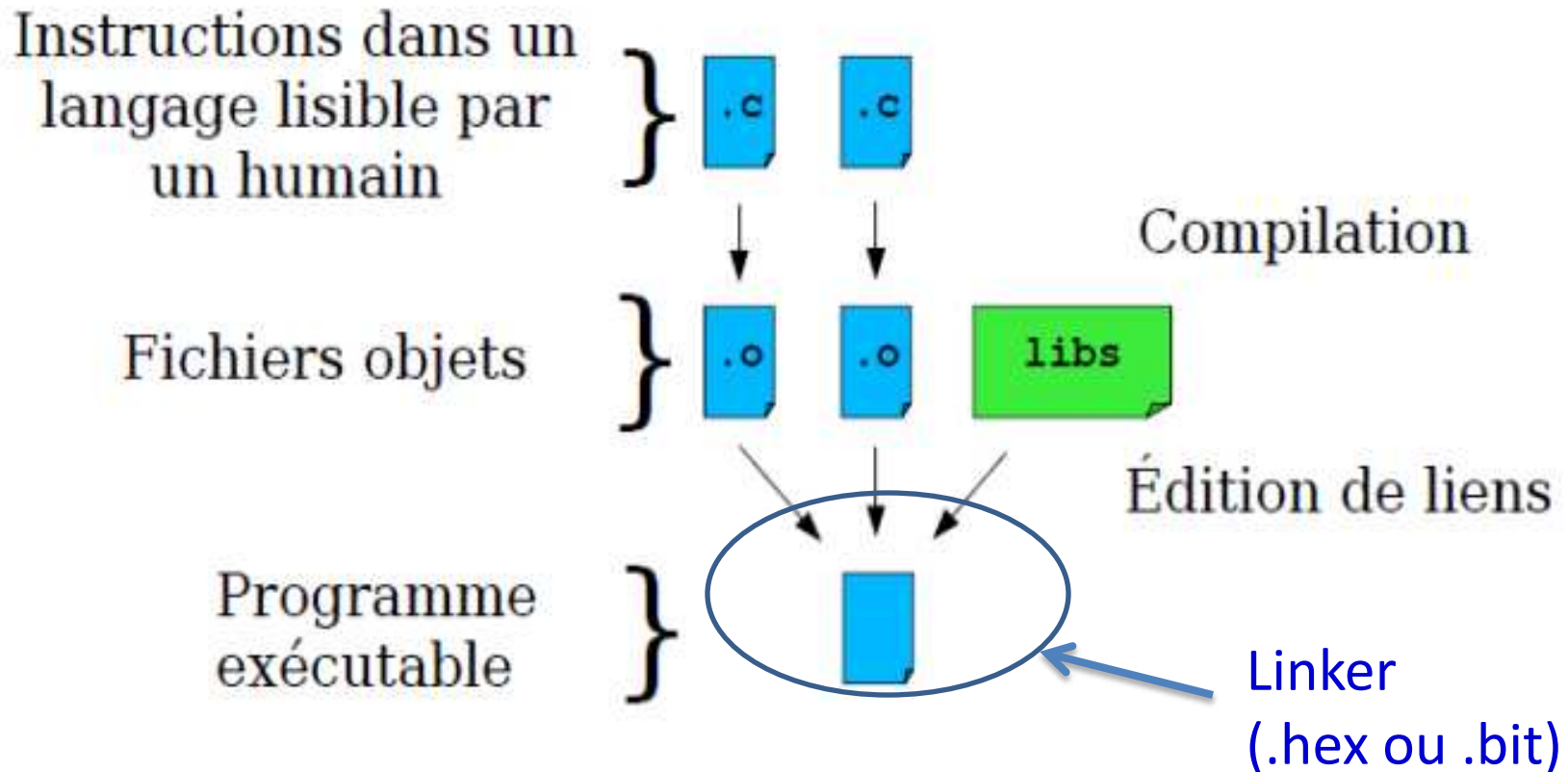
II- Les notions de base de la programmation en C

Remarques

- **Les commentaires** sont utilisés pour des raisons de **lisibilité et de compréhension** du programme. Un commentaire est une chaîne de caractères comprise entre **/* et */**, ou bien une ligne précédé par **//**
- Cette chaîne est **ignorée** par le compilateur C et peut déborder sur plusieurs lignes.
- Le langage C distingue les minuscules, des majuscules.
- Les mots réservés (exemples : **for, while, return, struct ...**) du langage C doivent être écrits **en minuscules**.

II- Les notions de base de la programmation en C

2- Compilation



II- Les notions de base de la programmation en C

3- Les variables

- Les variables permettent de représenter les **données d'un programme**.
- Chaque variable est matérialisée par un emplacement **mémoire identifié** par un **nom**.
- Chaque variable possède :
 1. un **nom**
 2. un **type**
 3. une **valeur**



**Une variable sera toujours définie avant utilisation !
Elle doit être associée à un type de données !**

II- Les notions base de la programmation en C

- C est un langage typé:

Types de base en C

Types Entiers

- **int** : entier standard
- **short** : entier court
- **long** : entier long
- **char** : caractère

Le langage C implémente le type char qui est en fait numérique.

Types Réels

- **float** : réel
- **double** : réel double
- **long double** : réel long double

II- Les notions de base de la programmation en C

Les variables/Exemple

main est toujours la fonction principale d'un programme C

Début de bloc d'instructions

Fin de bloc d'instructions

```
#include <stdio.h> /* Bibliothèque d'entrées-sorties
standard */

int main()
{
    int a;
    //déclaration

    a=127;
    //affectation

    printf("la valeur de a = %d",a); //affichage de
    la valeur de a
}
```

II- Les notions de base de la programmation en C

Les variables: Type (processeur ARM Cortex M4)

Type	Nb d'octets	Minimum	Maximum
int	4	-2^{31}	$2^{31} - 1$
unsigned int	4	0	$2^{32} - 1$
short	2	-32768	32767
unsigned short	2	0	65535
long	4	-2^{31}	$2^{31} - 1$
unsigned long	4	0	$2^{32} - 1$
char	1	-128	127
unsigned char	1	0	255
float	4	-3.4×10^{38}	3.4×10^{38}
double	8	-1.7×10^{308}	1.7×10^{308}
long double	10	-3.4×10^{4932}	1.7×10^{4932}

II- Les notions base de la programmation en C

4- Les opérateurs de base

- **Opérateurs arithmétiques :**

- **+** : addition $\Rightarrow a + b$
- **-** : soustraction $\Rightarrow a - b$
- ***** : multiplication $\Rightarrow a * b$
- **/** : division (entière et rationnelle) $\Rightarrow a / b$
 - Si a et b sont des entiers ($a = 7; b = 2$) a / b donne 3
 - Si a ou b est un réel ($a = 7; b = 2.0$) a / b donne 3.5
- **%** : modulo (reste de la division entière) $\Rightarrow a \% b$
 - $a = 7; b = 2$ alors $a \% b$ donne 1

II- Les notions de base de la programmation en C

Les opérateurs logiques

Opérateur	Name	Notation
&	AND	X&Y
	OR	X Y
~	NOT	~X
>>	Right shift	X>>Y
<<	Left shift	X<<Y
^	XOR	X^Y

Pas de type booléen en C Les opérateurs logiques considèrent toute valeur **différente de zéro** comme vrai et **zéro** comme faux.

(4 < 10)

1 (vrai)

!(5 > 1)

0 (faux)

II- Les notions base de la programmation en C

Les opérateurs de comparaison

- Opérateurs de comparaison : (1: vrai ou 0: faux)
 - `==` : égal à $\Rightarrow a == b$
 - `!=` : différent de $\Rightarrow a != b$
 - `<` : strictement inférieur $\Rightarrow a < b$
 - `<=` : inférieur ou égal $\Rightarrow a <= b$
 - `>` : strictement supérieur $\Rightarrow a > b$
 - `>=` : supérieur ou égal $\Rightarrow a >= b$

II- Les notions base de la programmation en C

Les opérateurs d'affectation

- Affectation simple
 - `int x;`
 - `x = 4;`
- Affectation simple
 - `int x;`
 - `x = 2;`
 - `x = x + 1;`
- Affectation combinée
 - `var1 = (var1) op (var2)    ⇔    var1 op= var2`
 - `i = i + 1;    ⇔    i += 1;`

II- Les notions de base de la programmation en C

Les opérateurs d'incrémentation / décrémentation

- Incréments et décréments:

- `++` : augmentation de 1 `x++` $\longleftrightarrow$ `x = x+1;`
;

- `y = x++;` $\longleftrightarrow$ `y = x; x = x+1;`

- `int x = 3;` •

y

 •

x

- `y = ++x` $\longleftrightarrow$ `x = x + 1; y = x;` •

x

 •

y

- `--` : diminution de 1

- `x--;` $\longleftrightarrow$ `x = x-1;`

- `y = x--;` $\longleftrightarrow$ `y = x; x = x-1;`

II- Les notions de base de la programmation en C

Affichage des données

- Fonction **printf()** : `printf (" text Format ", NomVal);`

Permet d'afficher des chaînes de caractères et des données de types différents

Format	Données
%d	Nombre entier
%u	Nombre entier non signé
%c	Caractère
%f	Nombre à virgule flottante

- Fonction **scanf()**: `Scanf ("Format", &NomVal);`

Permet de lire des chaînes de caractères et traite des données de types différents

II- Les notions de base de la programmation en C

Afficher le contenu des variables

Format	Type attendu
%d	int
%ld	long
%f	float
%f	double

```
3  int nombreDeVies = 5, niveau = 1;
4
5  printf("Vous avez %d vies et vous etes au niveau n° %d\n", nombreDeVies, niveau);
```

Récupérer une saisie

```
1  int age = 0;
2  scanf("%d", &age);
```

II- Les notions de base de la programmation en C

5- Structures conditionnelles

L'instruction <<if ... else>>

```
if (Condition)
```

```
{  
  ...  
  ...  
}
```

Bloc
«if»

```
else
```

```
{  
  ...  
  ...  
}
```

Bloc «else»
Optionnel

Condition=Expression logique
simple ou combinée.

Exemples: (A<B), (M>10.0), (Delta <=0),
((Age > 18) && (Moy>= 14.0))

Traitement à effectuer **si la condition est vraie** (toute valeur $\neq 0$). À la fin du traitement, il y a saut après la partie else.

Traitement à effectuer **si la condition est fausse** (valeur = 0).

II- Les notions de base de la programmation en C

Exemple

```
85         break;
86     default :   printf("\nERREUR DE CHOIX");
87     // pas de break ici
88 }
89 }
90
91 #include <stdio.h>
92 void main()
93 {
94     float moy;
95     printf ("Donnez votre moyenne: \n");
96     scanf ("%f",&moy);
97
98     if ( moy >= 10 )
99
100         printf ("Mabrouk ! Vous avez réussi.");
101
102     else
103         printf ("Echec: Il faut travailler encore plus!");
104 }
105
106
```

II- Les notions de base de la programmation en C

Les <<if imbriqués>>

```
if (condition 1)
    liste d'instructions 1
else
    if (condition 2)
        liste d'instructions 2
    else
        if (condition 3)
            liste d'instructions 3
        else
            if (condition N)
                liste d'instructions N
            else
                liste d'instructions N+1
```

II- Les notions de base de la programmation en C

Exemple

```
53  #include <stdio.h>
54  void main()
55  {
56  int A,B;
57  printf("Entrez deux nombres entiers :");
58  scanf("%i %i", &A, &B);
59  if (A > B)
60      printf("%i est plus grand que %i\n", A, B);
61  else
62      if (A < B)
63          printf("%i est plus petit que %i\n", A, B);
64      else
65          printf("%i est égal à %i\n", A, B);
66  }
```

II- Les notions de base de la programmation en C

L'instruction <<switch>>

```
switch (expression)
{
    case val1 :    ...
                    ...
                    break;
    case val2 :
                    break;
    case val3 :
                    ...
                    break;
    case val4 :
                    ...
                    break;
    default :      ...
}
```

Uniquement les expressions entières sont permises: int, short, long ou char.

Val1, val2, ... doivent être des constantes du même type que Expression.

Obligatoire pour ne pas entrer dans le «case» suivant

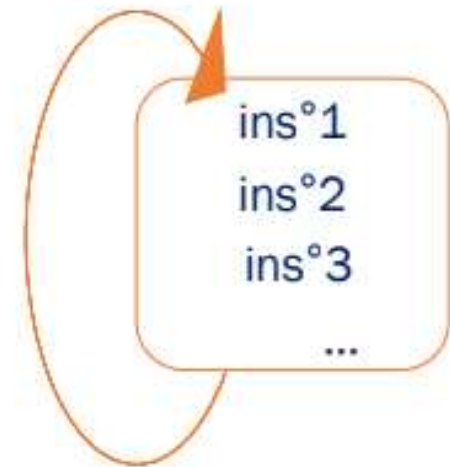
Exécuté lorsque aucun des «cases» précédents n'est vrai

II- Les notions de base de la programmation en C

6- Les structures répétitives

- Une ou plusieurs instructions qui **se répètent**:

Un cas
simple



- Il existe **2 familles** de boucles en langage C:
 1. **for** (nombre d'itérations connu à l'avance)
 2. **while** et **do ... while** (nombre d'itérations non connu à l'avance).

II- Les bases de base de la programmation en C

Exemple

```
/* Programme pour la boucle for */  
#include <stdio.h>  
void main ()  
{  
    int a, k=10;  
  
    for (a=1; a <= k; a++)  
        printf (\'\'Bonjour\'');  
}
```

TD Langage C

Exercice 1: Mon premier programme C

Écrivez un programme en langage C qui affiche le message "Bonjour, monde !" sur la console.

Exercice 2: Affichage de variables

Déclarez une variable entière `int` et initialisez-la avec votre âge. Déclarez une variable `double` et initialisez-la avec votre taille en mètres. Affichez ces variables sur la console avec des messages descriptifs.

Exercice 3: Somme de deux nombres

Écrivez un programme qui demande à l'utilisateur d'entrer deux nombres entiers, puis affiche leur somme.

Exercice 4: Calcul de moyenne

Demandez à l'utilisateur d'entrer trois nombres décimaux. Calculez et affichez leur moyenne.

Exercice 5: Nombre pair ou impair

Écrivez un programme qui demande à l'utilisateur un nombre entier et détermine s'il est pair ou impair.

Exercice 6: Le plus grand de deux nombres

Demandez à l'utilisateur d'entrer deux nombres. Affichez le plus grand des deux.

TD Langage C

Exercice 7: Type de triangle

Demandez à l'utilisateur d'entrer les longueurs des trois côtés d'un triangle. Déterminez et affichez s'il s'agit d'un triangle équilatéral, isocèle ou scalène.

Exercice 8: Année bissextile

Écrivez un programme qui vérifie si une année donnée par l'utilisateur est une année bissextile. Une année est bissextile si elle est divisible par 4, sauf si elle est divisible par 100 mais pas par 400.

II- Les notions de base de la programmation en C

La boucle <<while>>

- Tant qu'une condition est vraie, répéter un bloc d'instructions.

```
while (condition)
{
    ...
    bloc d'instructions
    ...
}
```

Expression logique simple ou combinée. Cette condition est testée avant chaque itération.

bloc d'instructions exécuté si la condition est vraie (0 ou n fois)

Si la boucle ne comporte qu'une instruction, les accolades ne sont plus nécessaires.

II- Les notions de base de la programmation en C

Règles à retenir

- Si **on connaît** à l'avance le nombre de fois que le traitement sera répété  **Boucle for**
- Si on **ne connaît pas** à l'avance le nombre de fois que le traitement sera répété:
 - Si le traitement doit être exécuté **au moins une fois**  **Boucle do...while**
 - Si le traitement peut ne pas être exécuté du tout (**0 fois**)  **Boucle while**

II- Les notions de base de la programmation en C

La boucle <<do ... while>>

```
do {  
    bloc d'instructions  
}  
while (condition);
```

- La structure **do..while** ressemble à la structure **while**, sauf que...
La structure **while** évalue la condition avant d'exécuter le bloc d'instructions, alors que la structure **do..while** évalue la condition après avoir exécuté le bloc d'instructions (à la fin de l'itération).



Avec la boucle **do..while** le bloc d'instructions est exécuté **au moins une fois**

II- Les notions de base de la programmation en C

7- Les fonctions

Définition

```
#include <stdio.h>
void NomFonction()
{
    .....
}
```

Fonction principale

```
void main()
{
    int a, b, valmax;
    printf(" Tapez deux entiers: ");
    scanf("%d %d", &a, &b);
```

Appel

```
NomFonction ();
```

```
.....
}
```

II- Les notions de base de la programmation en C

- Syntaxe

```
Type_de_retour nomDeFonction (type paramètre1, type  
    paramètre2,..... )  
{  
    déclaration;  
    instruction1;  
    ....  
    return résultat;  
}
```

II- Les notions de base de la programmation en C

- Exemple:

TYPE de la valeur de retour: un réel

2 entiers comme paramètres

Nom de la fonction

```
float evaluerExp (float a, float b)
{
    float val=0;
    val=2*(a+b);
    return val;
}
```

Les instructions

Valeur renvoyée

The diagram illustrates the components of a C function definition. It shows the function signature `float evaluerExp (float a, float b)` and the function body enclosed in curly braces. Annotations with arrows point to specific parts: 'TYPE de la valeur de retour: un réel' points to the `float` return type; '2 entiers comme paramètres' points to the parameters `float a` and `float b`; 'Nom de la fonction' points to the function name `evaluerExp`; 'Les instructions' points to the two lines of code inside the function body; and 'Valeur renvoyée' points to the `return val;` statement.

II- Les notions de base de la programmation en C

Appel des fonctions

- L'appel des fonctions dépend de type de retour :

- Cas 1: La fonction n'a pas de code retour:

- Syntaxe:

nomDeFonction (param1, param2..);

- Cas 2: La fonction a un code retour:

- Syntaxe:

val = nomDeFonction(param1, param2..);

Où « **val** » est une variable de même type que le type de retour de la fonction.

III- Les pointeurs

1- Les paramètres et leur mode de transmission

Il ya deux types de paramètres:

- **Paramètres formels** : utilisés lors de définition de la fonction
- **Paramètre effectifs** : utilisés lors de l'appel de la fonction

```
#include <stdio.h>
int cube(int x){
    return x * x * x;
}

void afficheCube(int x){
    int cube = cube(x);
    printf("%d \n", cube);
}

int main(){
    int res = cube(3);
    printf("%d \n", res);
    afficheCube(4);
    return 0;
}
```

Paramètres formels

Paramètres effectifs

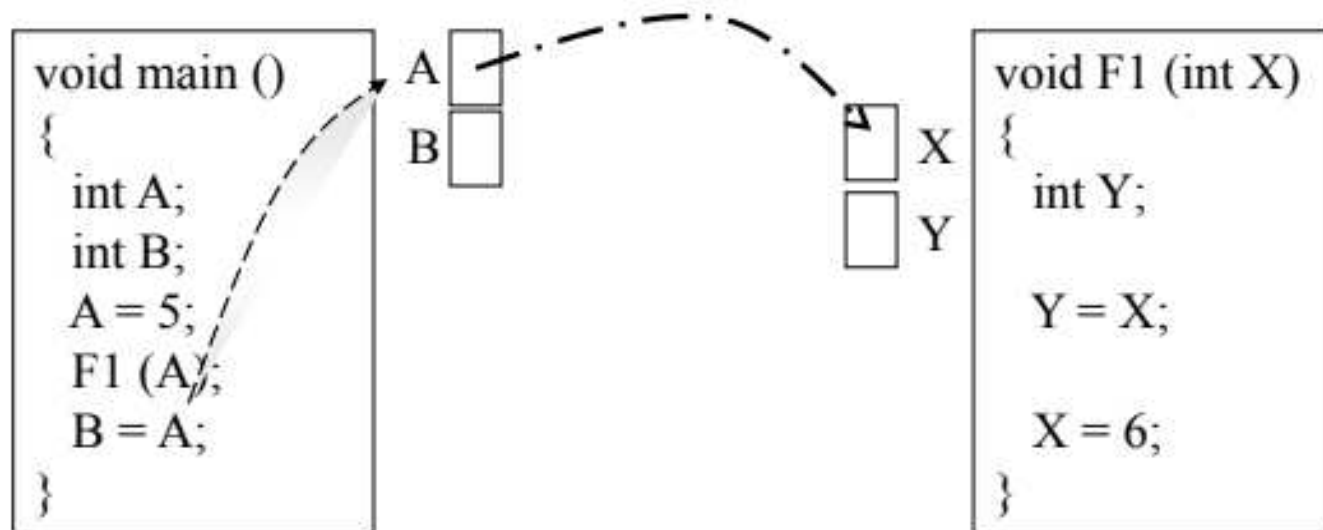
Il ya deux types de passage de paramètre:

- **Le mode par copie de valeur**
- **Le mode par copie d'adresse (utilise les pointeurs)**

III- Les pointeurs

Passage par copie de valeur

- Copie des valeurs des paramètres effectifs dans les paramètres formels



C'est la transmission de paramètres par copie de valeur

III- Les pointeurs

Passage par copie de valeur

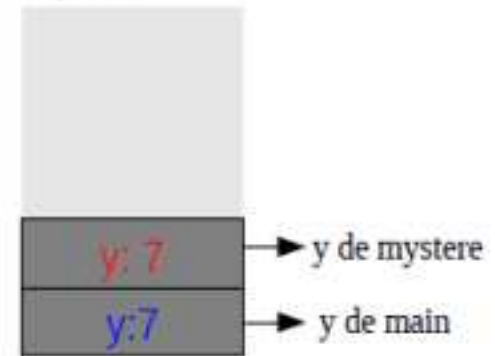
- Que se passe-t-il ?

```
#include <stdio.h>
void mystere(int y) {
    y = 3;
}
int main() {
    int y = 7;
    printf("%d \n", y);
    mystere(y);
    printf("%d \n", y);
    return 0;
}
```

1 - Avant mystere:



2 - Entrée dans mystere:



3 - avant de sortir de mystere:



4 - Après mystere :



III- Les pointeurs

2- Utilité des pointeurs

- On peut accéder aux données en mémoire à l'aide de pointeurs i.e. des variables pouvant contenir des adresses d'autres variables.
- Les pointeurs sont le seul moyen de changer le contenu de variables déclarées dans d'autres fonctions.
- Les pointeurs nous permettent d'écrire des programmes plus compacts et plus efficaces.

III- Les pointeurs

3- Manipulation des pointeurs

- Le symbole **NULL** permet d'initialiser un pointeur qui ne pointe sur rien. Cette valeur NULL peut être affectée à tout pointeur quel que soit le type.
- Le symbole NULL est défini dans la librairie **stdlib.h**.
- Exemple:

```
int *pNombre=NULL;
```

Dès qu'on déclare un pointeur, il est préférable de l'initialiser à NULL

Attribuer une valeur à un pointeur

- Pour attribuer une valeur à un pointeur, il faut le faire pointer sur une variable précise.
- Exemple:

```
int A;  
int* p1;  
int* p2;  
p1=&A; /* p1 contient l'adresse de A */  
p2=p1; /* copie le contenu de p1 dans p2 → p2 et p1 pointent sur la  
même variable A*/
```

- L'opérateur & permet d'obtenir l'adresse d'une variable.
- L'opérateur & ne peut pas être appliqué à des constantes ou des expressions.
- p1 et p2 désignent une variable pointeur initialisée à l'adresse de la variable A de type int.

III- Les pointeurs

Accès au contenu d'un pointeur

- L'opérateur *
- Pour avoir accès au contenu d'un pointeur, on utilise l'opérateur * suivi du nom du pointeur.
- Exemple:

```
int *pNombre;  
int n=20;  
pNombre=&n; /* pNombre pointe sur l'entier n qui contient  
la valeur 20 */  
printf ( ' ' %d ' ', *pNombre); /*affiche la valeur 20 */  
*pNombre=40;  
printf( ' ' %d ' ', *pNombre); /*affiche la valeur 40 */  
printf( ' ' %d ' ', n); /*affiche la valeur 40 */
```

III- Les pointeurs

3- Fonctions et pointeurs

- En C, `return` ne permet de retourner qu'une seule valeur.
- Parfois une fonction doit retourner plusieurs résultats en sortie.
- Il n'y a qu'une solution pour retourner plusieurs résultats: passer en paramètre l'adresse des variables où seront stockés les résultats.
- Le passage par adresse est aussi utile pour modifier le contenu de variables déclarées dans d'autres fonctions .

III- Les pointeurs

Exemple

- Une fonction retourne le minimum et le maximum de deux entiers a et b.

```
void min_max(int a, int b, int * pmin, int* pmax)
{
    if(a<b) {        *pmin=a;
                    *pmax=b;
    } else {
        *pmin=b;
        *pmax=a;
    }
}

int main()
{
    int a,b, min,max;
    printf("'Entrer deux entiers'");
    scanf("%d %d", &a,&b);
    min_max(a,b,&min,&max);
    printf("le min est %d le max est %d",min,max);
}
```

- Pour retourner plusieurs résultats, la fonction n'a qu'à prendre l'adresse des variables ou stocker les résultats

TD Langage C

Exercice 9: Compte à rebours

Écrivez un programme qui affiche un compte à rebours de 10 à 1 en utilisant une boucle `while`, puis affiche "Décollage!".

Exercice 10: Somme des N premiers entiers

Demandez à l'utilisateur un entier `n`. Calculez et affichez la somme des `n` premiers entiers positifs en utilisant une boucle `for`.

Exercice 11: Table de multiplication

Demandez à l'utilisateur un nombre entier. Affichez la table de multiplication de ce nombre de 1 à 10 en utilisant une boucle `for`.

Exercice 12: Factorielle d'un nombre

Calculez la factorielle d'un nombre entier positif donné par l'utilisateur en utilisant une boucle `while` ou `for`.

- Le tableau permet de désigner **sous un seul nom** un ensemble **de valeurs de même type**, repérées par des indices.
- Un enregistrement, appelé structure En langage C, est une variable complexe qui permet de désigner **sous un seul nom** un ensemble de valeurs pouvant être **de type différent**.

V- Tableaux et structures

Les Tableaux

int T[5];

Type des éléments

Nom

Nombre d'éléments



```
int Premiers [4];
```

```
Premiers [0] = 2;
```

```
Premiers [1] = 3;
```

```
Premiers [2] = 5;
```

```
Premiers [3] = 7;
```

```
printf("Le 3e nombre premier: %d\n", Premiers[2]);
```

```
printf("Somme du 2e et du 3e: %d", Premiers[1]+ Premiers[2]);
```

En C:

1. Les indices commencent par 0
2. T[i] : i+1ème élément



Le 3e nombre premier: 5
Somme du 2e et du 3e: 8

V- Tableaux et structures

Les Structures

Syntaxe:

```
struct <Nom_Structure> {  
  <type_champ1> <Nom_Champ1>;  
  <type_champ2> <Nom_Champ2>;  
  <type_champ3> <Nom_Champ3>;  
  ...  
};
```

Exemple:

```
struct etudiant {  
  char nom[30];  
  int age;  
  float moyenne;  
};
```

Déclaration

```
struct <Nom_Structure> <Var_Structure>;  
struct etudiant E1;
```

Syntaxe:

```
struct <Nom_Structure> {  
  <type_champ1> <Nom_Champ1>;  
  <type_champ2> <Nom_Champ2>;  
  <type_champ3> <Nom_Champ3>;  
  ...  
};
```

Typedef <Nom_Structure> <nom_Definition>

Exemple:

```
struct etudiant {  
  char Nom[30];  
  int Age;  
  float Moyenne;  
};
```

Typedef struct etudiant **Etudiant**;

Déclaration

```
<Nom_Définition> <Var_Structure>;  
Etudiant E1;
```

Utilisation d'une structure

- Soit **E1** et **E2** deux variables de type **Etudiant**:
- Il existe deux types d'affectation:
 - Soit d'une façon individuelle (champs par champs), sur chacun de leurs champs
 - **E1.niveau = E2.niveau;**
 - **E1.moyenne = E2.moyenne;**
 - Soit d'une manière globale sur toute la structure.
 - **E1 = E2;**

V- Tableaux et structures

Utilisation d'une structure

- Accès aux champs:
 - Pour accéder aux champs d'une structure on utilise l'opérateur de champ (**un simple point**) placé entre le nom de la variable structurée et le nom du champ.
 - Syntaxe: `<Var_Structure>.<Nom_Champi>;`
- Pour affecter des valeurs à la variable **E1**, on pourra écrire:
 - `E1.Niveau = 2; // affectation d'un champs`
 - `printf ("donner la moyenne \n ");`
 - `scanf ("%f", &E1.moyenne); //stockage dans un champs`

Structure contenant des tableaux

- Une structure peut contenir des champs de type chaîne de caractères ou bien de type tableau.

- Exemple:

```
struct Etudiant
{
    char nom [30];
    char prenom [30];
    float notes [4];
};
struct Etudiant E1;
```

- Notation:

- E1.nom[0] : désigne le premier caractère du champ nom de l'étudiant E1
- E1.notes[3] : désigne la quatrième note du tableau notes de l'étudiant E2

TD Langage C

Exercice 13: Initialisation et affichage d'un tableau

Déclarez un tableau d'entiers de taille 5. Initialisez-le avec les valeurs 10, 20, 30, 40, 50. Affichez tous les éléments du tableau.

Exercice 14: Somme des éléments d'un tableau

Écrivez un programme qui calcule et affiche la somme de tous les éléments d'un tableau d'entiers donné.

Exercice 15: Recherche du plus grand élément

Trouvez et affichez le plus grand élément d'un tableau d'entiers.

Exercice 16: Inversion d'un tableau

Écrivez un programme qui inverse l'ordre des éléments d'un tableau d'entiers.